

CS6868 Research Mini-Project

Concurrent Union-Find in OCaml 5 Using CAS Atomics

Yaswanth Srinivas
cs23b047@smail.iitm.ac.in

Prince
cs23b045@smail.iitm.ac.in

Mukesh
cs23b040@smail.iitm.ac.in

April 30, 2026

Abstract

We implement and evaluate a lock-free-style concurrent union-find (disjoint-set union, DSU) in OCaml 5 using `Atomic.compare_and_set`. Four variants share a common DSU module interface: a sequential baseline, a coarse-grained mutex, a per-node lock, and the CAS implementation. On a workload of 200 000 nodes and 1 000 000 interleaved operations, CAS peaks at four OCaml domains with a $1.22\times$ speedup for 20% unions and a $1.76\times$ speedup for 50% unions. CAS failure rates remain below 0.1% at all domain counts, indicating that the performance ceiling is caused by atomic memory traffic and scheduler overhead rather than by retry storms.

1 Goals

The goal of this project was to implement and evaluate concurrent union-find in OCaml 5. Union-find maintains a partition of a fixed element set and supports `find` (return a component representative), `union` (merge two components), and `same_set` (test connectivity). The structure is a good fit for a concurrency project because small writes to shared parent pointers determine global connectivity, so correctness depends on careful synchronisation rather than just wrapping a sequential algorithm.

Research question. Does a CAS-based concurrent union-find in OCaml 5 outperform lock-based alternatives on realistic interleaved graph-connectivity workloads, and how does graph structure (density, degree distribution) affect CAS contention and speedup?

2 Background

Sequential union-find represents each component as a rooted tree in which every element stores a parent index (roots point to themselves). Union-by-rank links the shallower tree under the deeper one, incrementing the winner's rank when both trees have equal rank. Path compression shortens future searches by rerouting nodes directly toward the root. Together these optimisations give near-constant amortised cost per operation [2].

In the concurrent setting every optimisation becomes a potential race. A coarse mutex serialises all operations: correct but unscalable. Per-node locks allow concurrent access to disjoint subtrees but require a fixed lock ordering to avoid deadlock. CAS uses optimistic synchronisation: read the current state, compute an update, and install it only if the field still holds the observed value; retry on failure.

The CAS approach suits union-find because the critical writes are narrow (one parent index, one rank integer) and failures are rare. Our implementation uses *path halving*: when traversing

from node x , if x 's grandparent g differs from its parent p , attempt `compare_and_set parent[x]` `p g` and continue from p regardless of whether the CAS succeeded. For `union`, the critical step is:

Listing 1: CAS link in `union` (`cas_union_find.ml`).

```
(* winner/loser chosen by rank, then by smaller index *)
if Atomic.compare_and_set t.parent.(loser) loser winner then (
  (* optional rank increment if equal-rank merge *)
  if same_rank && Atomic.get t.parent.(winner) = winner then begin
    let r = Atomic.get t.rank.(winner) in
    ignore (Atomic.compare_and_set t.rank.(winner) r (r + 1))
  end;
  true)
else
  attempt () (* loser is no longer a root; retry *)
```

The linearisation point of a successful `union` is the CAS that sets `parent[loser] := winner`. A failed CAS means another domain has already merged or compressed that root; the operation simply retries with fresh roots. A failed rank increment does not break correctness because ranks are only a balancing heuristic.

3 Tasks Undertaken

3.1 Implementation

All four implementations satisfy the DSU module type in `union_find.mli`, which exposes `create`, `find`, `union`, `same_set`, `component_count`, `stats`, and `reset_stats`. This made tests and benchmarks implementation-agnostic.

`sequential_union_find.ml` `int array` for parent and rank; full path compression (two-pass, extra cache misses on the return pass) and union-by-rank; no synchronisation. Ground-truth reference for all cross-checks.

`mutex_union_find.ml` Single `Mutex.t` wrapping the whole structure. Every operation acquires it on entry. Trivially correct; eliminates all parallelism.

`node_lock_union_find.ml` One `Mutex.t` per node; `Atomic.t` arrays for parent and rank. `find` reads atomically without locking. `union` locks both roots in ascending index order (deadlock prevention) and re-checks they are still roots before linking. Key subtlety: same-set must be checked *before* acquiring locks — otherwise two domains on the same component deadlock trying to acquire the same lock pair.

`cas_union_find.ml` `int Atomic.t array` only; no locks. Uses *path halving* (single forward pass, skips every other node) — avoids the reverse write-back cache misses of full compression and fits naturally into a CAS loop. Failed CAS $\Rightarrow$ retry from fresh roots; no domain ever blocks another [1].

Table 1 summarises the key design choices across the four variants.

	Sequential	Mutex	Node-lock	CAS
Storage	int array	int array	int Atomic.t array	int Atomic.t array
Synchronisation	none	1 global lock	1 lock per node	lock-free CAS
Blocking?	n/a	yes	yes	no
Path compression	full	halving	halving	halving
Thread-safe	no	yes	yes	yes

Table 1: Design comparison of the four union-find implementations.

Pseudocode for the three concurrent implementations

The three listings below show the essential structure of each concurrent `find` and `union` in pseudocode. Low-level details (stats counters, `component_count` updates) are omitted for clarity.

Listing 2: Global-mutex `find` and `union`.

```
(* One Mutex.t for the entire structure *)
find t x:
  lock(t.mutex)
  root <- follow parent[] until parent[root] = root
  (* full path compression: repoint all nodes to root *)
  compress t.parent x root
  unlock(t.mutex)
  return root

union t x y:
  lock(t.mutex)
  rx <- find_nolock t x; ry <- find_nolock t y
  if rx = ry then (unlock; return false)
  link by rank (loser.parent := winner)
  unlock(t.mutex)
  return true
```

Listing 3: Per-node-lock `find` and `union`.

```
(* One Mutex.t per node; Atomic.t arrays for parent and rank *)
find t x:
  (* read-only atomic traversal, no locks *)
  while parent[x] != x:
    x <- Atomic.get parent[x]
  return x

union t x y:
  loop:
    rx <- find t x; ry <- find t y
    if rx = ry then return false
    (* lock both roots in ascending index order to prevent deadlock *)
    lo, hi <- min(rx,ry), max(rx,ry)
    lock(node_lock[lo]); lock(node_lock[hi])
    (* re-check: another domain may have merged these roots already *)
    if parent[lo] != lo || parent[hi] != hi then
      unlock both; goto loop
    link by rank (loser.parent := winner)
    unlock both
    return true
```

Listing 4: CAS `find` (path halving) and `union`.

```

(* int Atomic.t arrays for parent and rank; no locks *)
find t x:
  (* path halving: one forward pass, skip every other node *)
  while true:
    p <- Atomic.get parent[x]
    if p = x then return x          (* x is root *)
    gp <- Atomic.get parent[p]
    if gp != p then
      CAS parent[x] p gp          (* try to skip p; ignore failure *)
    x <- p

union t x y:
  loop:
    rx <- find t x; ry <- find t y
    if rx = ry then return false
    winner, loser <- choose by rank, then by smaller index
    if CAS parent[loser] loser winner then
      (* try to increment winner's rank if equal-rank merge *)
      if same_rank && Atomic.get parent[winner] = winner then
        CAS rank[winner] r (r+1)  (* failure is safe: rank is a hint *)
      return true
    (* CAS failed: loser was already merged by another domain *)
    goto loop

```

3.2 Testing

Correctness was checked at several levels. All four test suites are run with `make test` (which calls `dune build first`); individual suites can be run with the targets below.

make correctness

```
dune exec ./Union_find_correctness_test.exe
```

Deterministic tests (empty/singleton structures, basic unions, repeated unions, self-unions, component counts, fixed graph examples, disconnected components, stats counters) run on all four implementations, followed by randomised cross-checks: 30 trials each of Erdős–Rényi, star, and disconnected graphs comparing each implementation's canonical partition against the sequential reference, parallel replay over four domains, and 20 high-contention stress trials.

make stm

```
dune exec ./QCheck_stm.exe
```

QCheck-STM models the DSU as a sequence of successful `union` calls and checks that `find/union/same_set` agree with the model. Runs 300 sequential agreement tests and 150 parallel agreement tests (with automatic shrinking on failure) for each of mutex, node-lock, and CAS.

make lin

```
dune exec ./QCheck_lin.exe
```

QCheck-Lin generates parallel two-domain histories and checks that some sequential interleaving of the observed responses is valid. Runs 200 linearisability tests per concurrent implementation over a small universe.

make cas-stress

```
dune exec ./cas_stress_test.exe
```

CAS-specific stress: 30 graph-stress trials (random-plus-star edges over four domains, partition checked against sequential replay) and 20 mixed-operation trials (four workers $\times$ 400 operations each, counters verified to be non-zero).

Running `make bench` executes the two performance benchmarks:

`dune exec ./union_find_bench.exe` Scaling benchmark across 1/2/4/8/16 domains for CAS, mutex, and node-lock; outputs `bench_trend_results.csv` and the two trend plots.

`dune exec ./union_find_graph_bench.exe` Graph-structure benchmark (sparse-random, dense-random, star) for CAS only; outputs `bench_graph_structure_results.csv` and `bench_graph_time.png`.

`dune exec ./union_find_connectivity_bench.exe` Connected-component labelling benchmark: partitions edges across domains, each domain calls `union` on its slice, then `component_count` is verified against a sequential reference. Five graph profiles (sparse-random, dense-random, star, grid, chain); outputs `bench_connectivity_results.csv` and `bench_connectivity_time.png`.

Testing shaped the implementation in several ways: it forced root re-validation after optimistic reads, correct handling of a failed CAS as a retry signal rather than an error, and partition comparisons in benchmarks rather than timing alone.

4 Evaluation

4.1 Experimental setup

Benchmarks used 200 000 nodes and 1 000 000 interleaved operations, with one warm-up run and three timed runs; the median is reported. Domains were varied over $\{1, 2, 4, 8, 16\}$. The machine was an Intel Core i5-10300H at 2.50 GHz with 4 physical cores and 8 hardware threads; 16-domain results are an oversubscription experiment. Two operation mixes were evaluated: 20% unions / 80% finds and 50% unions / 50% finds. Operations within each run were interleaved rather than split into a build phase and a query phase, because separated phases suppress union/find contention.

4.2 Results

Figures 1 and 2 show the runtime curves for the two mixes. Table 2 summarises the best observed median and speedup from `bench_trend_results.csv`.

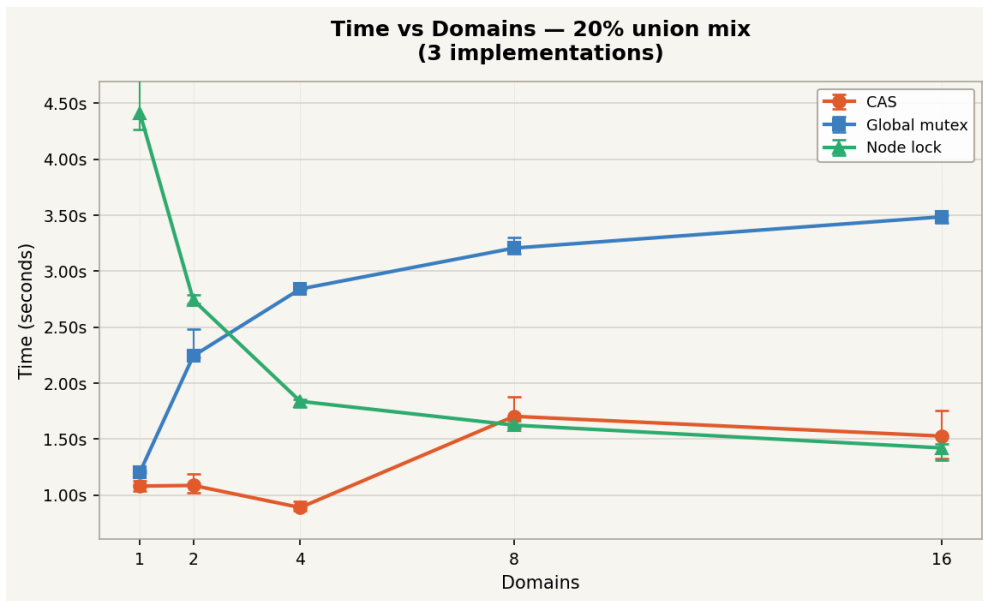


Figure 1: Median runtime — 20% union / 80% find workload.

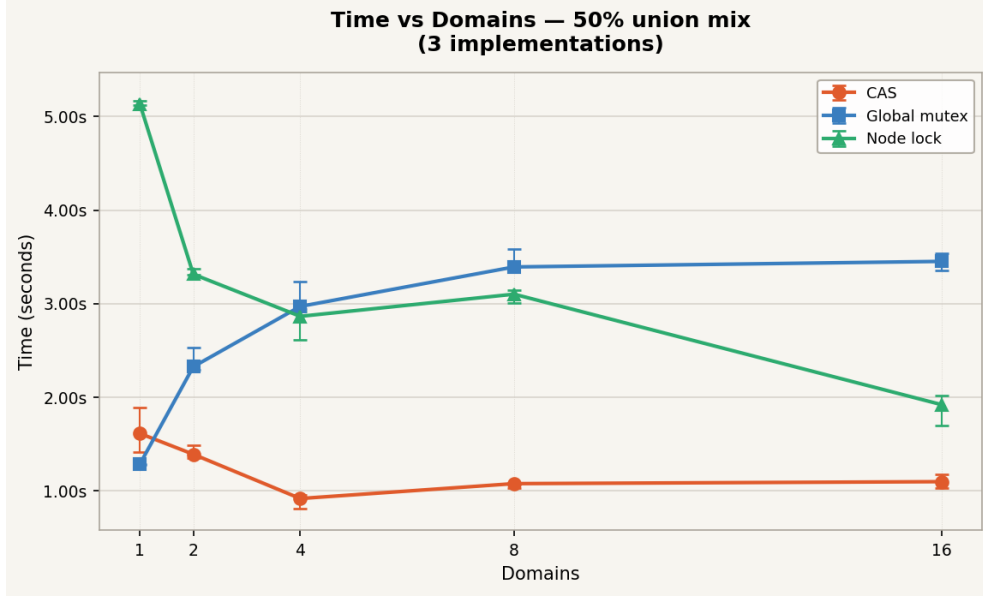


Figure 2: Median runtime — 50% union / 50% find workload.

Workload	Implementation	1 domain	Best time	Best domains	Speedup
20% union	CAS	1.081 s	0.889 s	4	1.22×
20% union	Mutex	1.205 s	1.205 s	1	—
20% union	Node-lock	4.413 s	1.421 s	16	3.11×
50% union	CAS	1.612 s	0.916 s	4	1.76×
50% union	Mutex	1.284 s	1.284 s	1	—
50% union	Node-lock	5.137 s	1.920 s	16	2.68×

Table 2: Best observed median runtime and speedup (speedup relative to own 1-domain time; mutex does not scale so no speedup is listed).

Mutex. The coarse mutex is competitive at one domain but degrades steadily as domains increase, reaching roughly $0.35\times$ the single-domain throughput at 16 domains for both mixes. The single lock serialises the entire operation sequence regardless of contention.

CAS. CAS achieves its best time at four domains for both mixes, matching the physical core count. Beyond four domains, runtime rises: the 20% workload slows from 0.889 s to 1.527 s at 16 domains; the 50% workload from 0.916 s to 1.096 s. However, CAS failure counters show that failures are not the cause: at 16 domains the 20% mix recorded only 43 failures out of $\sim 373\,000$ CAS attempts, and the 50% mix 359 out of $\sim 388\,000$ (both below 0.1%). The performance ceiling is therefore caused by atomic memory traffic, cache-line movement, OCaml domain scheduling, and hardware oversubscription rather than by retry storms.

Node-lock. Per-node locks are slow at one domain because every path-traversal step acquires a mutex, but they scale well in relative terms as independent graph regions can be processed in parallel. Despite the strong relative speedup, the absolute best times (1.421 s and 1.920 s) are worse than the best CAS times, so fine-grained locking pays a high constant overhead that CAS avoids.

Graph structure. Figure 3 isolates the CAS implementation across three graph profiles. Sparse-random graphs (low contention) show the clearest benefit at four domains (1.188 s $\rightarrow$ 0.838 s). Dense-random and star/hot-set profiles are faster at one domain but scale no better beyond four, because the star workload concentrates unions on a small set of representatives, causing repeated root contention rather than useful path compression.

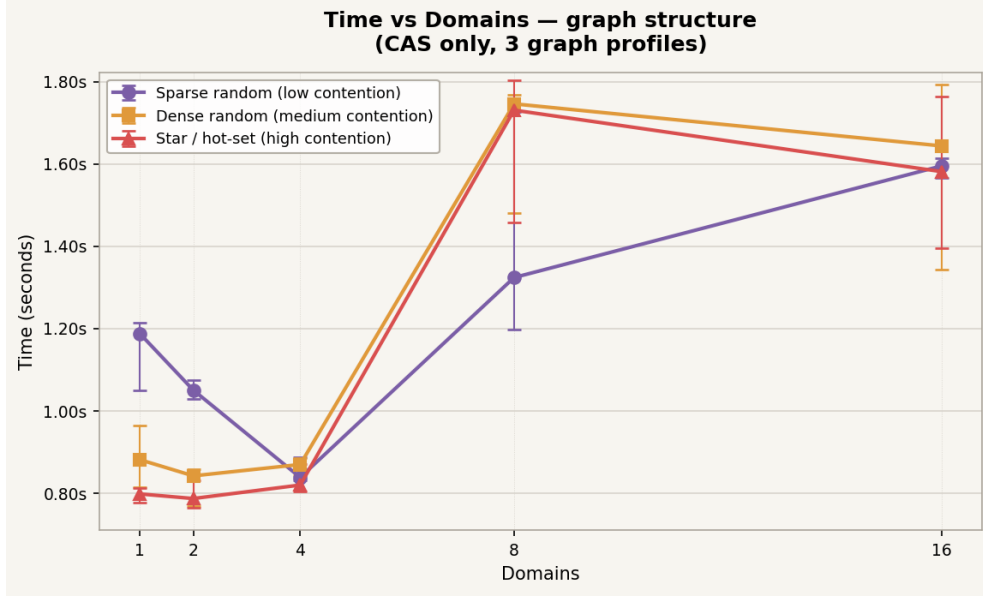


Figure 3: CAS runtime across three graph profiles (sparse random, dense random, star).

4.3 Connected-component labelling benchmark

`union_find_connectivity_bench.exe` evaluates a pure union-only workload: build a graph, partition its edges across d domains, let each domain call `union` on its slice, then read `component_count`. The final count is verified against a sequential reference run. This isolates the cost of *processing edges* from the mixed find/union traffic of the earlier benchmarks. Five graph profiles are tested; Figure 4 shows the results.

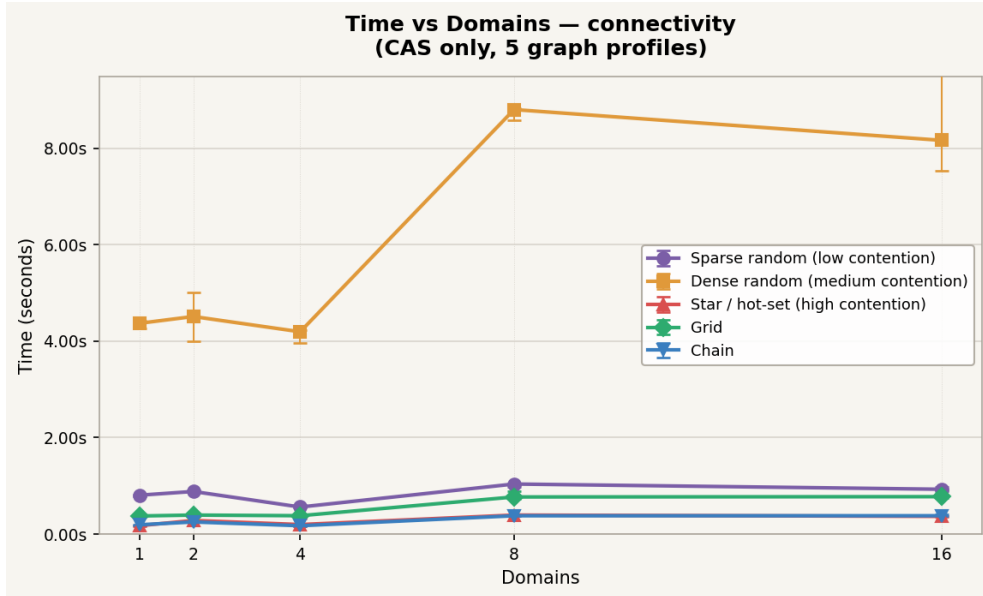


Figure 4: CAS connected-component labelling across five graph profiles.

Why dense-random is much slower. Dense-random uses average degree 40 (vs. degree 4 for sparse-random), giving 4 000 000 edges against $\approx 400\,000$ for all other profiles. Every edge still requires two `find` calls and a potential CAS, so the work is simply $10\times$ larger. Beyond that, once the dense graph collapses into a single component early in the run, every subsequent edge triggers a redundant connectivity check: both endpoints find the same root, no CAS is

attempted, but the full path-halving traversal is still paid. The result is $\sim 4\text{--}9\text{ s}$ versus $\lesssim 1\text{ s}$ for all other profiles.

Scaling by profile.

- **Sparse-random, grid** — moderate scaling up to 4–8 domains; disjoint subtrees allow genuine parallelism.
- **Star** — near-flat or slightly improving; all unions race to set the same root (node 0), so contention is high but path lengths are trivially short, keeping absolute time low ($\lesssim 0.2\text{ s}$).
- **Chain** — flat across all domain counts; edges are sequential by nature (0–1, 1–2, $\dots$), so partitioning gives each domain a contiguous run with almost no cross-partition contention.
- **Dense-random** — rises sharply from 4 to 8 domains ($4.2\text{ s} \rightarrow 8.8\text{ s}$) then levels off; the graph is already fully connected after the first few thousand edges, so extra domains spend most of their time on redundant finds with no useful union work to parallelise.

5 Reflection on the Use of LLMs

We used two LLMs during this project: **GPT-4.5** (via Codex, used primarily inside the editor for code assistance) and **Claude Sonnet 4** (used for design discussions, debugging explanations, and report drafting).

What worked. Both models were effective at generating boilerplate: module wrappers, stats counter scaffolding, CSV summary printers, and QCheck property skeletons. Claude suggested that interleaved operation arrays give a fairer concurrency benchmark than separated union and find phases (separated phases suppress union/find contention), which we adopted. GPT-4.5 helped untangle recursive lock-acquisition code and suggested the ascending-index locking discipline for the node-lock implementation.

What did not work. Generated explanations were sometimes overconfident about lock-free progress guarantees. Our implementation uses optimistic CAS updates validated by tests, not a formal proof of lock-freedom; we had to be careful not to inherit that framing. The models also predicted that higher domain counts would produce more CAS retries and proportionally slower runtimes, which the failure counters contradicted — failures stayed below 0.1% even at 16 domains.

A concrete bug caught without LLM help. GPT-4.5 helped with code involving locks and recursion, but it missed a subtle deadlock in an early draft of the node-lock `union`: the code acquired locks on both roots without first checking whether they were already in the same set. Two domains calling `union` on nodes from the same component would each identify the same two roots, then race to acquire the same pair of locks, and deadlock. The fix — check `find(x) = find(y)` before acquiring any lock, and re-verify root identity after acquiring — had to be reasoned through manually from first principles.

What was difficult. The hard reasoning was not mechanical: deciding what each operation is allowed to conclude from an evolving parent forest, identifying safe retry points, and understanding why a failed path-halving CAS is harmless (the path still shortens on the next access) whereas a failed union-link CAS requires a full retry from fresh roots. These required careful reading of test failures and invariant analysis that the LLMs could assist with but not replace.

6 Conclusions

A CAS-based union-find in OCaml 5 is correct under the tested concurrent histories and outperforms both lock-based alternatives at low to moderate domain counts. Its peak is at four domains (matching the physical core count), where it achieves speedups of $1.22\times$ and $1.76\times$ for 20% and 50% union mixes respectively. Beyond four domains, performance declines despite negligible CAS failure rates, pointing to atomic memory traffic and domain scheduling as the bottleneck on this hardware.

The per-node-lock design achieves stronger relative speedup but worse absolute time; the coarse mutex neither scales nor competes. Future work includes a formal linearisability argument, experiments on a larger many-core machine, and embedding the DSU inside a real graph algorithm such as parallel Kruskal’s MST.

Contributions

Member	Contribution (%)	Main responsibilities
Yaswanth Srinivas	33%	Benchmark harness, plots, evaluation, report writing
Prince	33%	Core implementation (CAS, mutex, node-lock, sequential)
Mukesh	33%	Explanation, analysis, testing, QCheck suites
Claude Sonnet 4	0.5%	Report drafting, design discussions, debugging explanations
GPT-4.5 (Codex)	0.5%	Boilerplate generation, lock-acquisition code assistance
Total	100%	

References

- [1] Dan Alistarh, Alexander Fedorov, and Nikita Koval. *In Search of the Fastest Concurrent Union-Find Algorithm*, volume 153 of *Leibniz International Proceedings in Informatics (LIPIcs)*. Schloss Dagstuhl – Leibniz-Zentrum für Informatik, Dagstuhl, Germany, 2020. URL: <https://drops.dagstuhl.de/entities/document/10.4230/LIPIcs.OPODIS.2019.15>, doi:10.4230/LIPIcs.OPODIS.2019.15.
- [2] Maurice Herlihy, Nir Shavit, Victor Luchangco, and Michael Spear. *The Art of Multiprocessor Programming*. Morgan Kaufmann, 2nd edition, 2020.